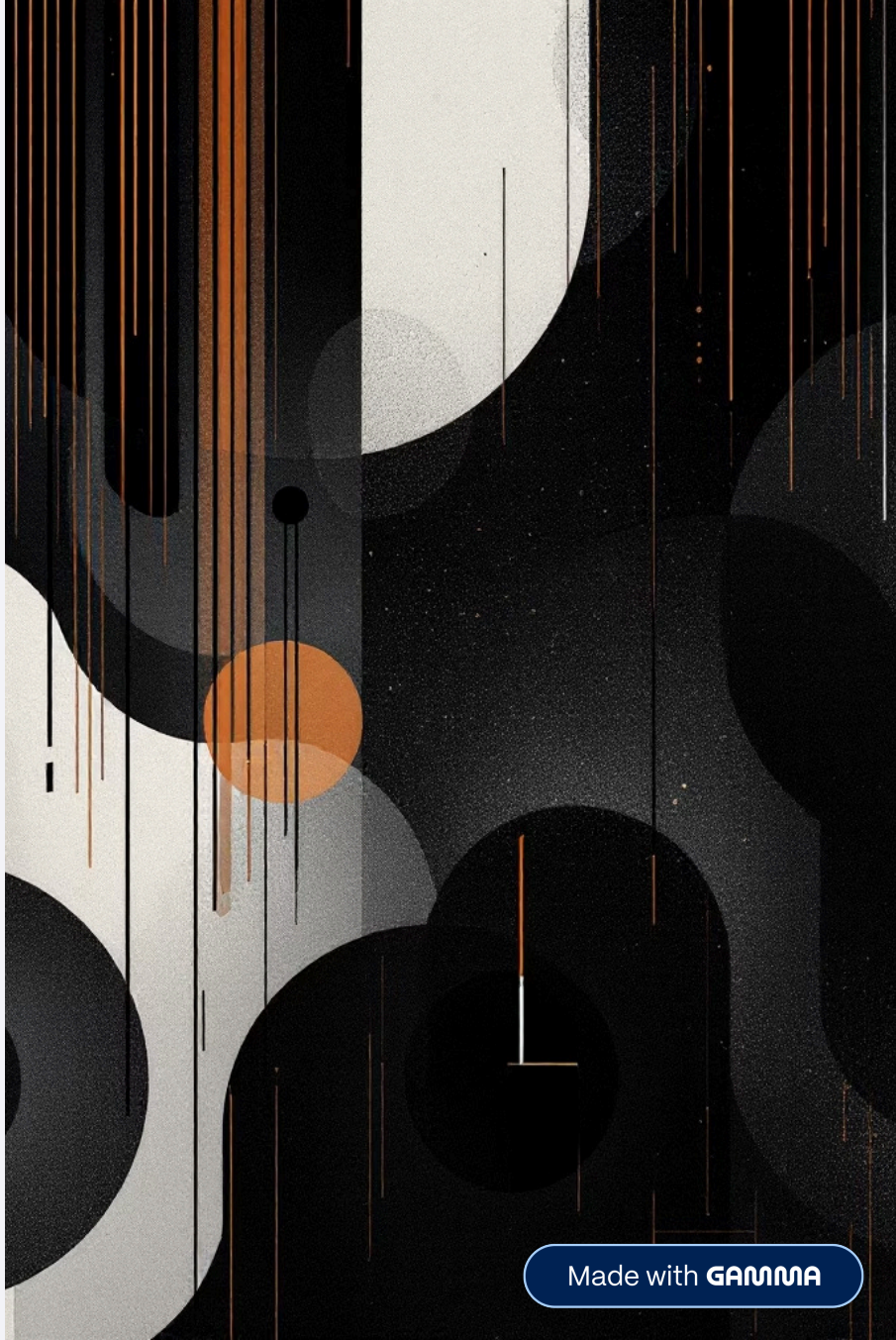
Fundamentos de Programação — PC.4 e PRJ.3

Uma jornada pelos fundamentos da Programação Orientada a Objetos e Algoritmos Recursivos



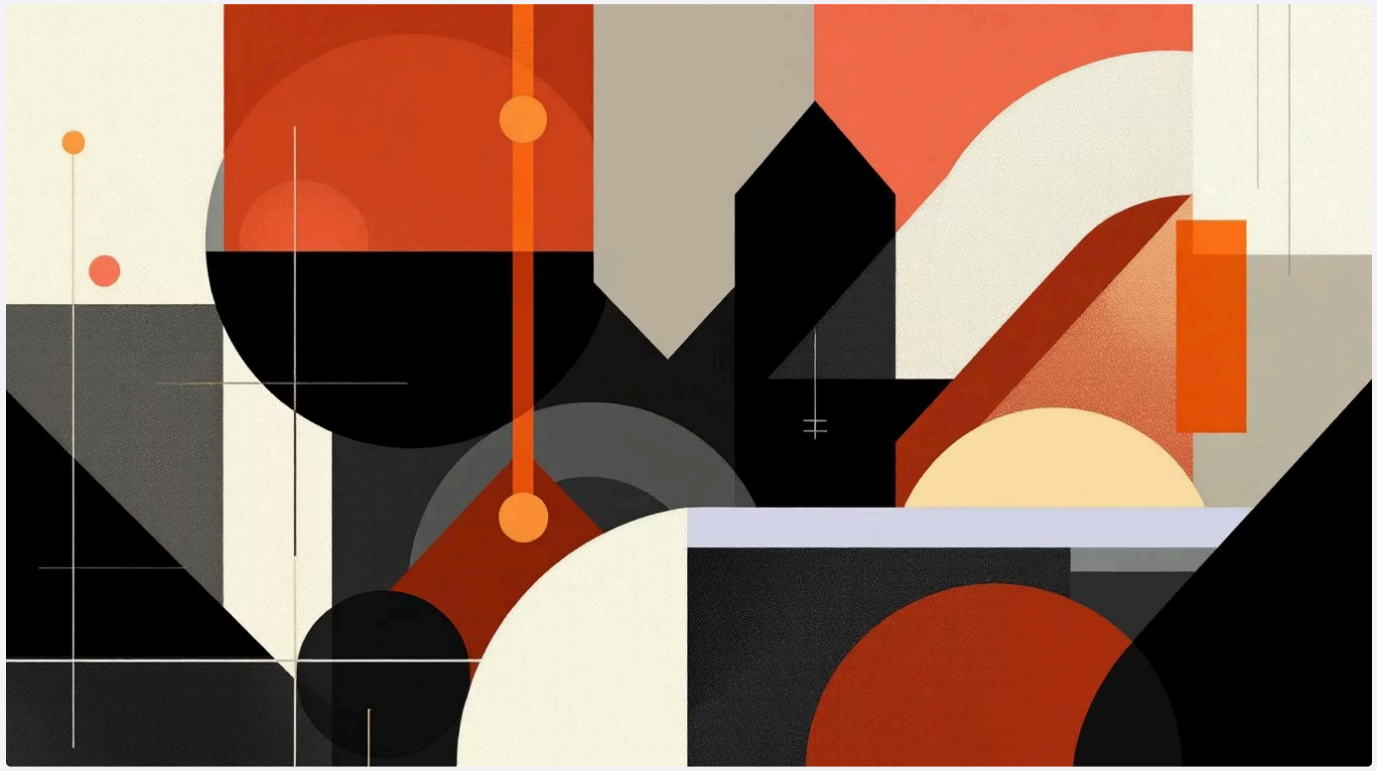
Made with GAMMA

Progressão Geométrica: Definição

Uma **Progressão Geométrica (PG)** é uma sequência numérica onde cada termo, a partir do segundo, é obtido multiplicando o termo anterior por uma constante chamada **razão (q)**.

Exemplo simples: **2, 6, 18, 54, 162...**

Na sequência acima, a razão é **q = 3**, pois cada termo é o anterior multiplicado por 3.



Made with GAMMA

Fórmula do Termo Geral

1

$a_n = a_1 \times q^{(n-1)}$

Fórmula fundamental da PG

2

a_1

Primeiro termo da sequência

3

q

Razão (constante multiplicativa)

4

n

Posição do termo desejado

Esta fórmula permite calcular qualquer termo da progressão sem precisar calcular todos os anteriores. Por exemplo, para encontrar o 5º termo de uma PG com $a_1 = 2$ e $q = 3$: $\mathbf{a_5 = 2 \times 3^4 = 2 \times 81 = 162}$.

Made with GAMMA

Os 4 Tipos de Progressão Geométrica

Tipo	Condição	Exemplo	Comportamento
Crescente	$q > 1$	2, 6, 18, 54...	Cresce infinitamente
Decrescente	$0 < q < 1$	64, 32, 16, 8...	Diminui gradualmente
Constante	$q = 1$	5, 5, 5, 5...	Mantém o valor
Alternante	$q < 0$	3, -6, 12, -24...	Alterna sinais

Made with GAMMA

Implementação Python com POO

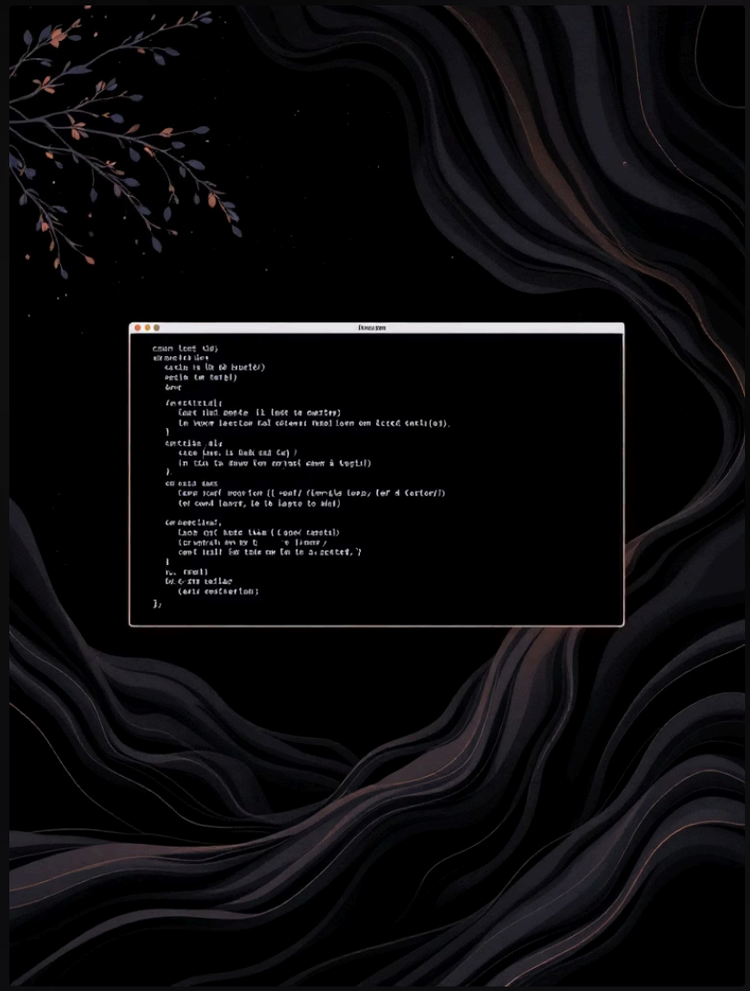
```python
class ProgressaoGeo:
    def __init__(self, primeiroTermo, razao):
        self.primeiroTermo = primeiroTermo
    def calcular_termo(self, n):
        if self.razao == 0: raise ValueError("Razão não pode ser zero")
        return self.primeiroTermo * (self.razao ** (n - 1))

    def calcular_soma(self, n):
        if self.razao == 1:
            return self.primeiroTermo * n
        return self.primeiroTermo * (self.razao ** n - 1) / (self.razao - 1)

    def exibir(self, n):
        for i in range(1, n + 1):
            print(self.calcular_termo(i))

pg = ProgressaoGeo(2,3)
pg.exibir(50)
```

Esta implementação utiliza **Programação Orientada a Objetos** para encapsular os dados e comportamentos relacionados à Progressão Geométrica em uma única classe reutilizável.



Made with GAMMA

Conceitos de POO Aplicados

Classe

Modelo que define atributos e métodos comuns. Aqui: **ProgressaoGeo**

Objeto

Instância específica da classe. Exemplo: **pg = ProgressaoGeometrica(2, 3)**

Atributo

Dados que caracterizam o objeto: **primeiroTermo** e **razao**

Método

Comportamentos/funcionalidades: **calcular_soma()** e **exibir()**

Encapsulamento

Agrupa dados e métodos, protegendo o estado interno do objeto

Made with GAMMA

Torres de Hanói: O Problema

Origem Histórica

Inventado pelo matemático francês **Édouard Lucas** em 1883. O quebra-cabeça consiste em três pinos e N discos de tamanhos diferentes.

Objetivo

Mover todos os discos do pino inicial para o pino destino, respeitando as regras.

Regra 1

Mover apenas um disco por vez

Regra 2

Nunca colocar disco maior sobre menor

Regra 3

Usar o pino auxiliar como intermediário

Made with GAMMA

Lógica Recursiva e Fórmula

1

Passo 1

Mover N-1 discos de A para B (usando C como auxiliar)

2

Passo 2

Mover o maior disco de A para C

3

Passo 3

Mover N-1 discos de B para C (usando A como auxiliar)

Caso Base: Quando N = 1, movemos diretamente o disco do pino origem para o destino.

Número Mínimo de Movimentos

$2^n - 1$

N Discos	Movimentos
1	1
2	3
3	7
4	15
5	31
6	63
7	127

Made with GAMMA

Código Recursivo Python

```python
def torre_de_hanoi(n, origem, destino, auxiliar):
    if n == 1:
        print("Mover disco 1 de", origem, "para", destino) #primeiro caso de recursão
        return
    torre_de_hanoi(n-1, origem, auxiliar, destino) #chamada recursiva
    print("Mover disco", n, "de", origem, "para", destino) #caso geral
    torre_de_hanoi(n-1, auxiliar, destino, origem) #chamada recursiva
torre_de_hanoi(3, 'A', 'B', 'C')
```

A função utiliza **recursão** para decompor o problema em subproblemas menores até atingir o caso base. Cada chamada recursiva reduz o número de discos a serem movidos.

Complexidade Computacional: $O(2^n)$

A complexidade **$O(2^n)$** indica **crescimento exponencial**. Para 64 discos, seriam necessários **$2^{64} - 1$** movimentos — aproximadamente 18 quintilhões de movimentos. Mesmo a 1 movimento por segundo, levaria mais de 584 bilhões de anos!

Made with GAMMA

Comparativo: PG vs Torres de Hanói

Aspecto	Progressão Geométrica	Torres de Hanói
Paradigma	Programação Orientada a Objetos (POO)	Algoritmo Recursivo
Fórmula Central	$a_n = a_1 \times q^{(n-1)}$	Movimentos = $2^n - 1$
Complexidade	O(n) para gerar N termos	$O(2^n)$ - Exponencial
Aplicações	Crescimento populacional, juros compostos, algoritmos de busca	Backup de dados, ordenação, teoria da computação

---

Ambos os conceitos demonstram a **importância de escolher o paradigma apropriado** para resolver problemas computacionais. A PG mostra como POO organiza dados e comportamentos, enquanto as Torres ilustram o poder (e limitações) da recursão para problemas exponenciais.

Made with GAMMA